TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

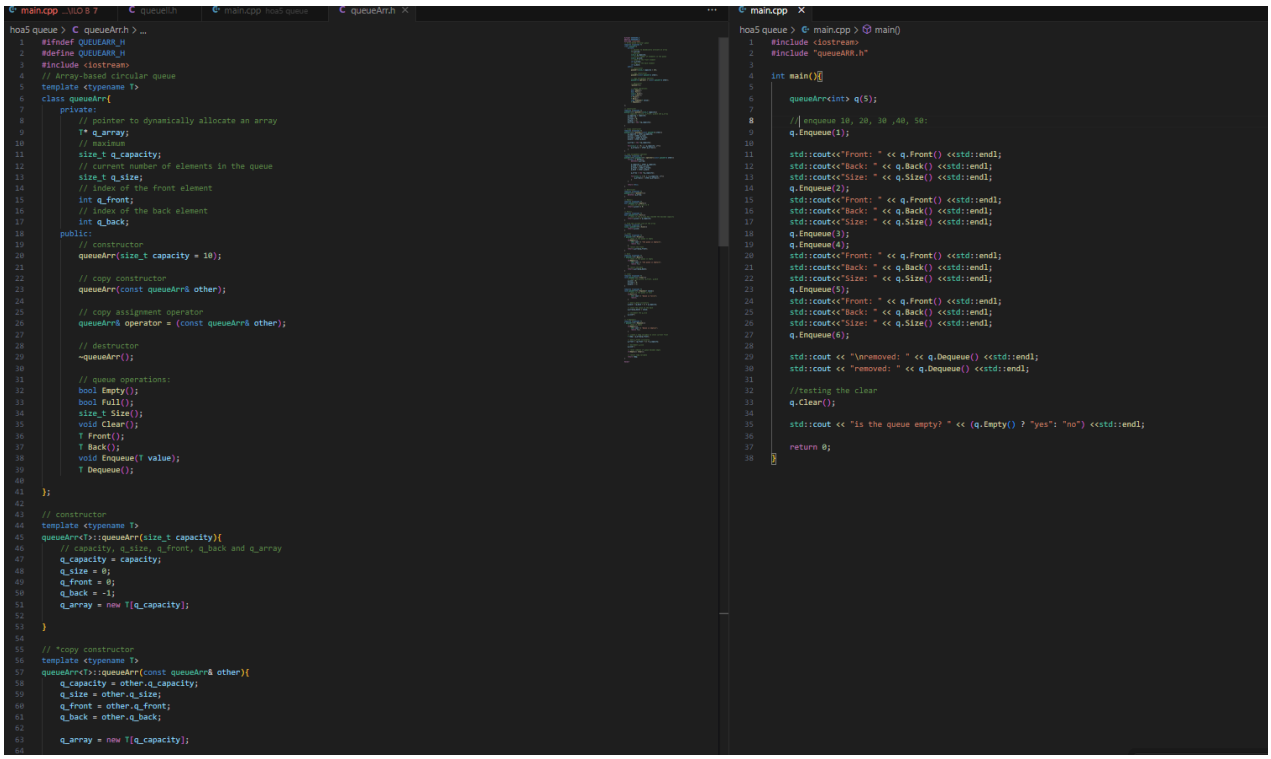
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structure and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	08/07/26	**DATE SUBMITTED:**	08/07/26
NAME:	Nieles, Cenndy M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __5.1_ Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?



```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
// Array-based circular queue
template <typename T>
class queueArr{
    private:
        // pointer to dynamically allocate an array
        T* q_array;
        // maximum
        size_t q_capacity;
        // current number of elements in the queue
        size_t q_size;
        // index of the front element
        int q_front;
        // index of the back element
        int q_back;
    public:
        // constructor
        queueArr(size_t capacity = 10);

        // copy constructor
        queueArr(const queueArr& other);

        // copy assignment operator
        queueArr& operator = (const queueArr& other);

        // destructor
        ~queueArr();

        // queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T Front();
        T Back();
        void Enqueue(T value);
        T Dequeue();

};

// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    // capacity, q_size, q_front, q_back and q_array
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];

}

// *copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new T[q_capacity];
```

```
#include <iostream>
#include "queueARR.h"

int main(){

    queueArr<int> q(5);

    // enqueue 10, 20, 30 ,40, 50:
    q.Enqueue(1);

    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(6);

    std::cout << "\nremoved: " << q.Dequeue() <<std::endl;
    std::cout << "removed: " << q.Dequeue() <<std::endl;

    //testing the clear
    q.Clear();

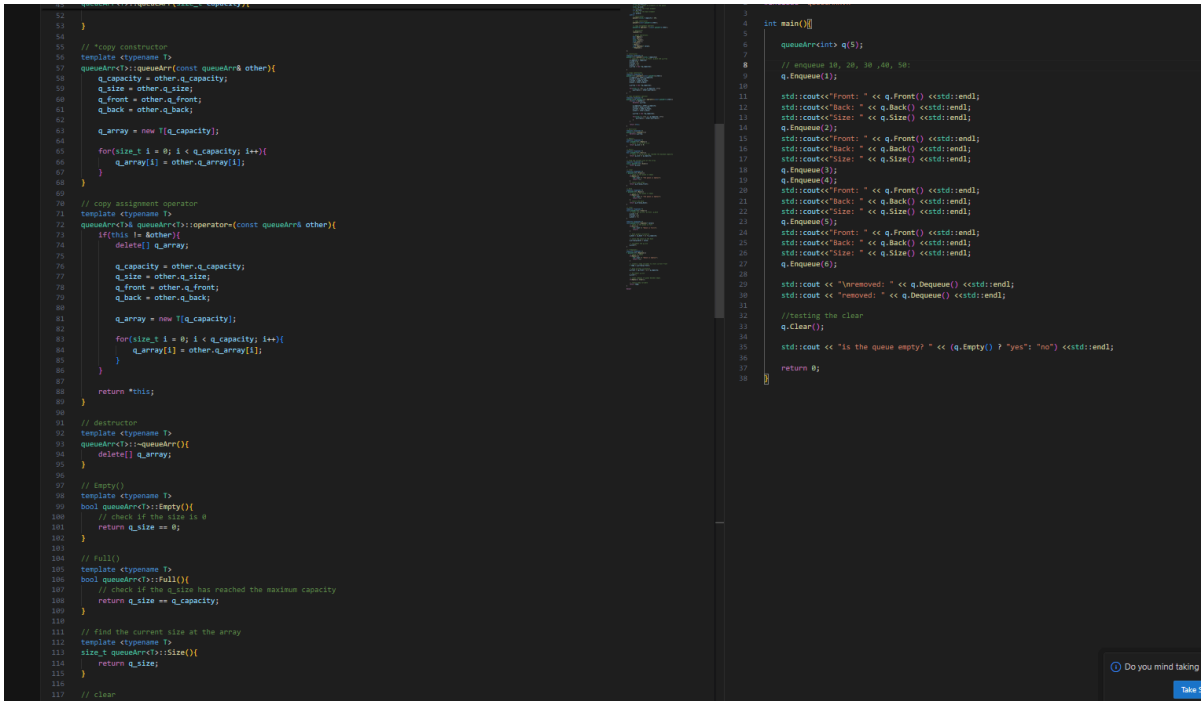
    std::cout << "is the queue empty? " << (q.Empty() ? "yes": "no") <<std::endl;

    return 0;
}
```

T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY



```cpp
    }

    // *copy constructor
    template <typename T>
    queueArr<T>::queueArr(const queueArr& other){
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new T[q_capacity];

        for(size_t i = 0; i < q_capacity; i++){
            q_array[i] = other.q_array[i];
        }
    }

    // copy assignment operator
    template <typename T>
    queueArr<T>& queueArr<T>::operator=(const queueArr& other){
        if(this != &other){
            delete[] q_array;

            q_capacity = other.q_capacity;
            q_size = other.q_size;
            q_front = other.q_front;
            q_back = other.q_back;

            q_array = new T[q_capacity];

            for(size_t i = 0; i < q_capacity; i++){
                q_array[i] = other.q_array[i];
            }
        }

        return *this;
    }

    // destructor
    template <typename T>
    queueArr<T>::~queueArr(){
        delete[] q_array;
    }

    // Empty()
    template <typename T>
    bool queueArr<T>::Empty(){
        // check if the size is 0
        return q_size == 0;
    }

    // Full()
    template <typename T>
    bool queueArr<T>::Full(){
        // check if the q_size has reached the maximum capacity
        return q_size == q_capacity;
    }

    // find the current size at the array
    template <typename T>
    size_t queueArr<T>::Size(){
        return q_size;
    }

    // clear
```

```cpp
int main(){

    queueArr<int> q(5);

    // enqueue 10, 20, 30 ,40, 50:
    q.Enqueue(1);

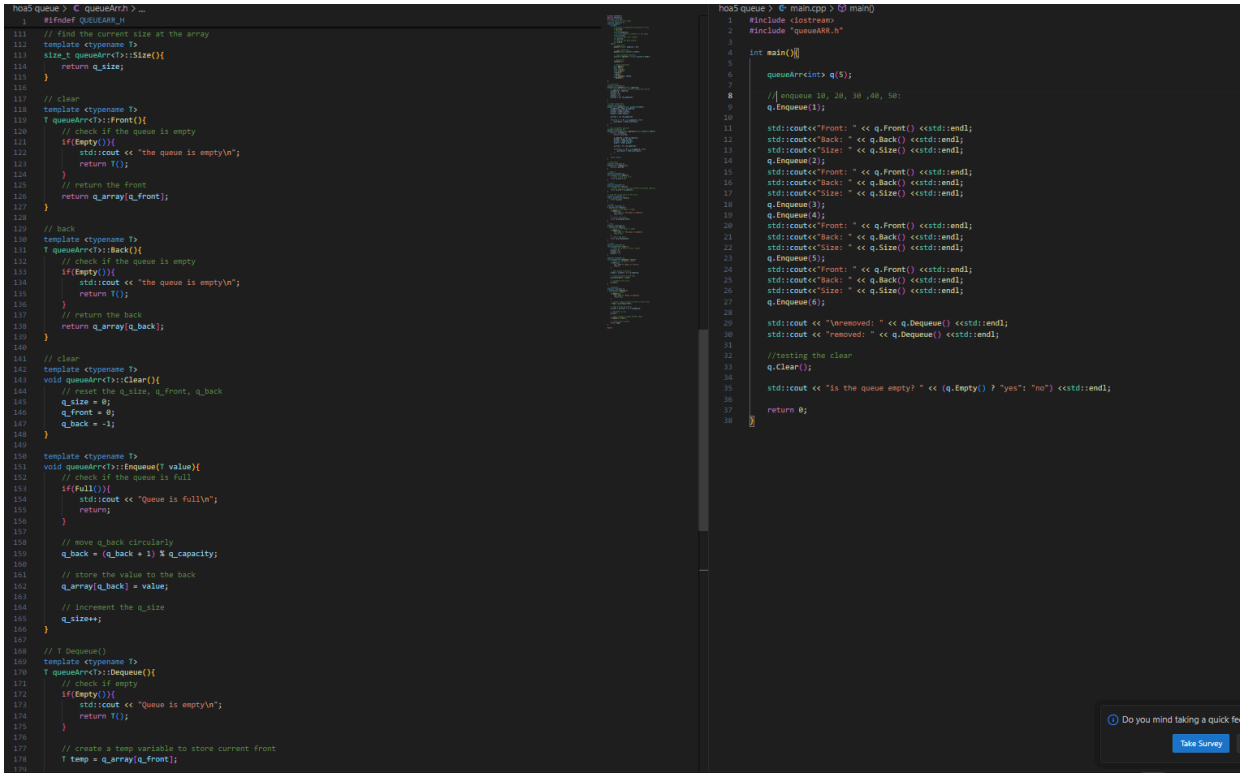
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(6);

    std::cout << "\nremoved: " << q.Dequeue() <<std::endl;
    std::cout << "removed: " << q.Dequeue() <<std::endl;

    //testing the clear
    q.Clear();

    std::cout << "is the queue empty? " << (q.Empty() ? "yes": "no") <<std::endl;

    return 0;
}
```



```cpp
#ifndef QUEUEARR_H
// find the current size at the array
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

// clear
template <typename T>
T queueArr<T>::Front(){
    // check if the queue is empty
    if(Empty()){
        std::cout << "the queue is empty\n";
        return T();
    }
    // return the front
    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back(){
    // check if the queue is empty
    if(Empty()){
        std::cout << "the queue is empty\n";
        return T();
    }
    // return the back
    return q_array[q_back];
}

// clear
template <typename T>
void queueArr<T>::Clear(){
    // reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template <typename T>
void queueArr<T>::Enqueue(T value){
    // check if the queue is full
    if(Full()){
        std::cout << "Queue is full\n";
        return;
    }

    // move q_back circularly
    q_back = (q_back + 1) % q_capacity;

    // store the value to the back
    q_array[q_back] = value;

    // increment the q_size
    q_size++;
}

// T Dequeue()
template <typename T>
T queueArr<T>::Dequeue(){
    // check if empty
    if(Empty()){
        std::cout << "Queue is empty\n";
        return T();
    }

    // create a temp variable to store current front
    T temp = q_array[q_front];
```

```cpp
#include <iostream>
#include "queueARR.h"

int main(){

    queueArr<int> q(5);

    // enqueue 10, 20, 30 ,40, 50:
    q.Enqueue(1);

    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " << q.Front() <<std::endl;
    std::cout<<"Back: " << q.Back() <<std::endl;
    std::cout<<"Size: " << q.Size() <<std::endl;
    q.Enqueue(6);

    std::cout << "\nremoved: " << q.Dequeue() <<std::endl;
    std::cout << "removed: " << q.Dequeue() <<std::endl;

    //testing the clear
    q.Clear();

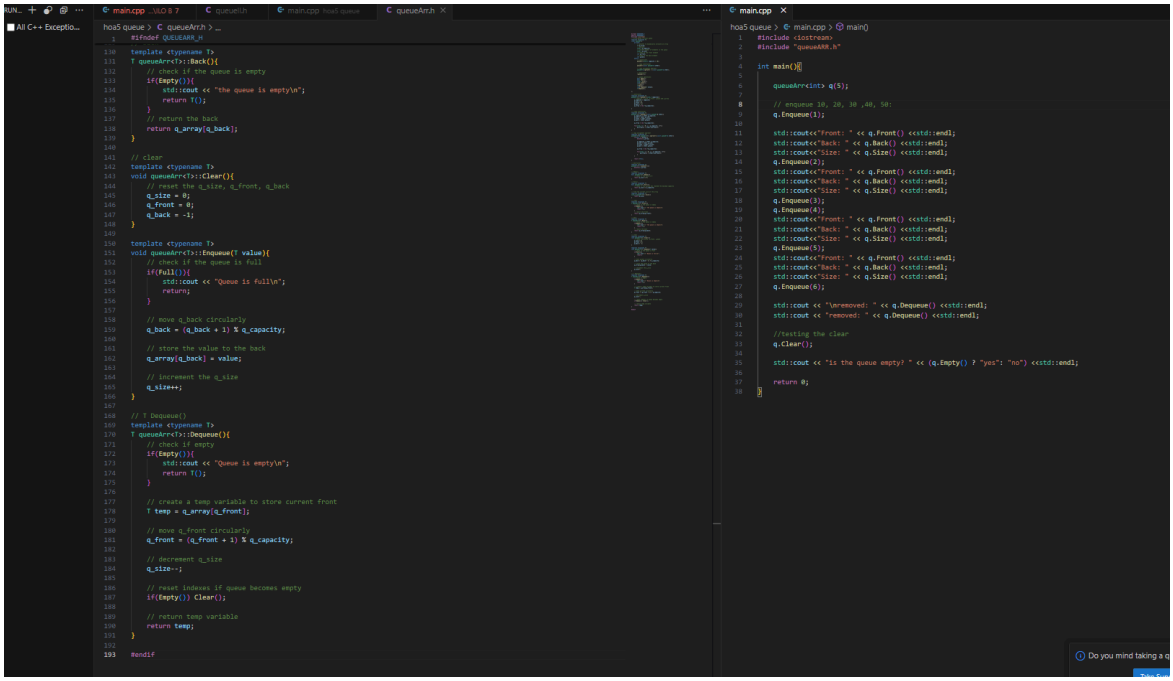
    std::cout << "is the queue empty? " << (q.Empty() ? "yes": "no") <<std::endl;

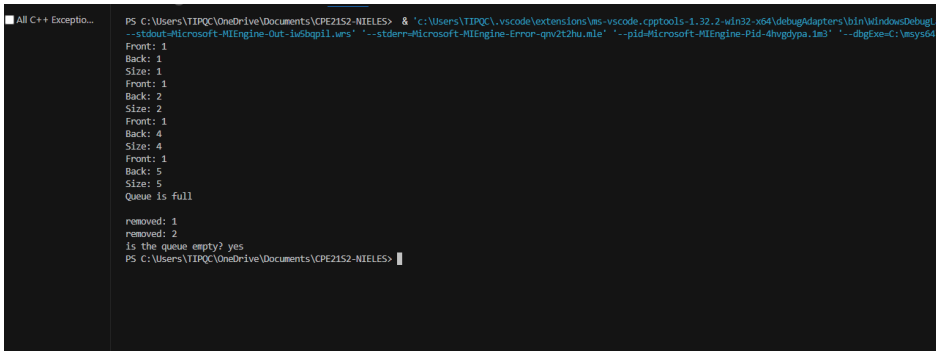
    return 0;
}
```

COMPUTER ENGINEERING

TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

QUEZON CITY



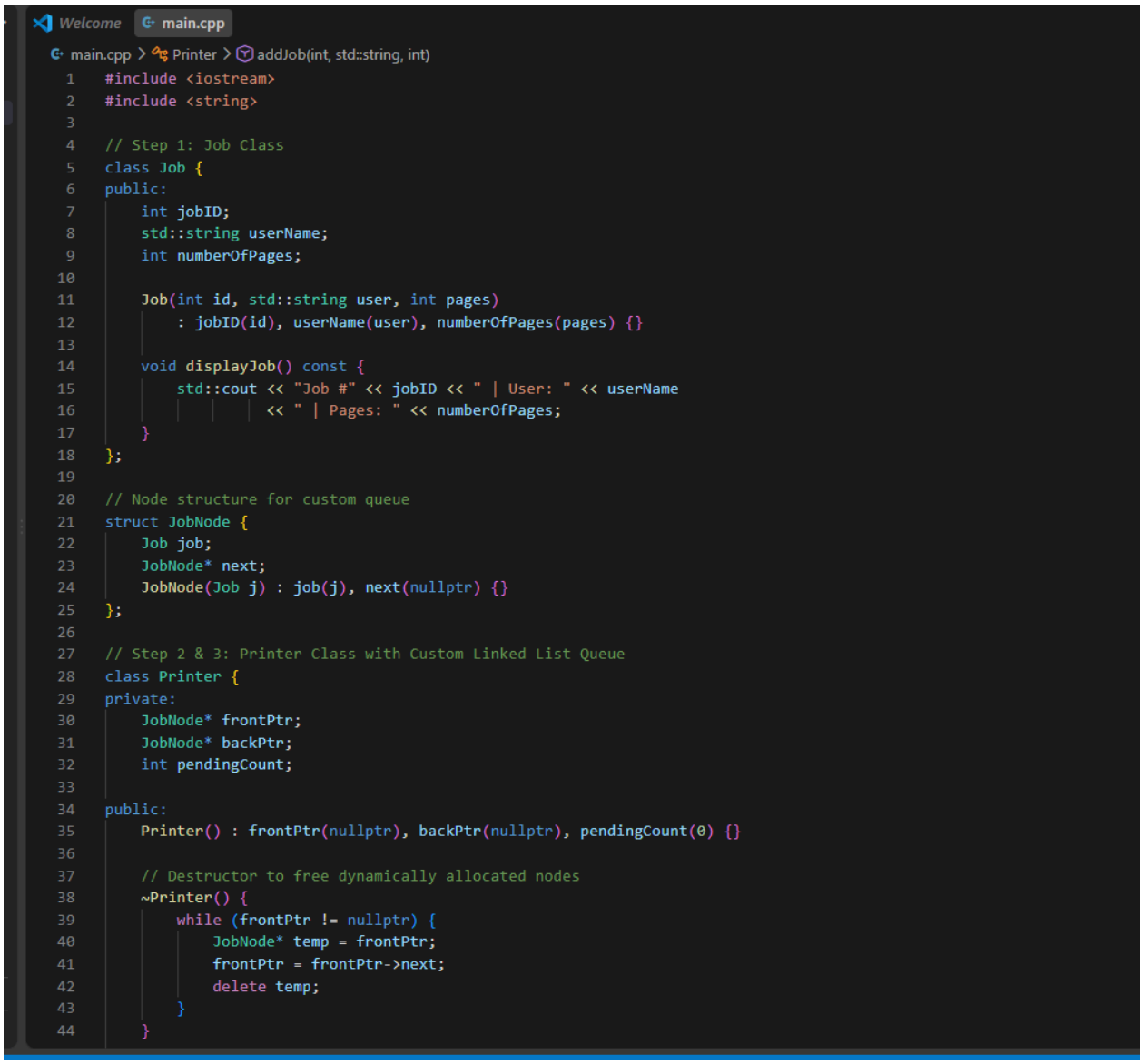


This activity implements an array-based circular queue using a VS Code the header is class named queArr.This header allows the queue to store different data types such as integers, character, and floating points number. The Queue follows the FIFO it means (First In, First Out) Which means the first element is also the first one to remove . The user dynamically allocated the (q_aaray) to store the queue elements.The variables q_capacity , q_size, q_front and q_back is used to manage the queue by setting its capacity and allocates the memory for the array.Overall the program of Queues showing how to demonstrate the concepts of the header, dynamically memory allocations,circular queue, constructors, the destructor operations. It provides the implementation of queues header.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using queue implementation



```cpp
#include <iostream>
#include <string>

// Step 1: Job Class
class Job {
public:
    int jobID;
    std::string userName;
    int numberOfPages;

    Job(int id, std::string user, int pages)
        : jobID(id), userName(user), numberOfPages(pages) {}

    void displayJob() const {
        std::cout << "Job #" << jobID << " | User: " << userName
                  << " | Pages: " << numberOfPages;
    }
};

// Node structure for custom queue
struct JobNode {
    Job job;
    JobNode* next;
    JobNode(Job j) : job(j), next(nullptr) {}
};

// Step 2 & 3: Printer Class with Custom Linked List Queue
class Printer {
private:
    JobNode* frontPtr;
    JobNode* backPtr;
    int pendingCount;

public:
    Printer() : frontPtr(nullptr), backPtr(nullptr), pendingCount(0) {}

    // Destructor to free dynamically allocated nodes
    ~Printer() {
        while (frontPtr != nullptr) {
            JobNode* temp = frontPtr;
            frontPtr = frontPtr->next;
            delete temp;
        }
    }
```

```
45
46      // Add a new print job to the queue
47      void addJob(int id, std::string user, int pages) {
48          Job newJob(id, user, pages);
49          JobNode* newNode = new JobNode(newJob);
50
51          if (frontPtr == nullptr) {
52              frontPtr = newNode;
53              backPtr = newNode;
54          }
55          else {
56              backPtr->next = newNode;
57              backPtr = newNode;
58          }
59          pendingCount++;
60
61          std::cout << "[ADDED] ";
62          newJob.displayJob();
63          std::cout << "\n";
64      }
65
66      // Process all jobs in First-Come, First-Served order
67      void processJobs() {
68          std::cout << "\n--- STARTING PRINTER JOB PROCESSING ---\n";
69          if (frontPtr == nullptr) {
70              std::cout << "No pending print jobs in the queue.\n";
71              return;
72          }
73
74          while (frontPtr != nullptr) {
75              JobNode* temp = frontPtr;
76              std::cout << "[PROCESSING] ";
77              temp->job.displayJob();
78              std::cout << " -> Completed!\n";
79
80              frontPtr = frontPtr->next;
81              delete temp;
82              pendingCount--;
```

```
        }
        backPtr = nullptr; // Reset the back pointer once queue is empty
        std::cout << "All print jobs completed successfully.\n";
    }

    // Display all jobs currently waiting
    void showPendingJobs() const {
        std::cout << "\n--- CURRENT PENDING JOBS (" << pendingCount << ") ---\n";
        if (frontPtr == nullptr) {
            std::cout << "Printer queue is empty.\n";
            return;
        }
        JobNode* curr = frontPtr;
        while (curr != nullptr) {
            std::cout << " -> ";
            curr->job.displayJob();
            std::cout << "\n";
            curr = curr->next;
        }
    }
};

// Step 4: Scenario Simulation
int main() {
    Printer officePrinter;

    std::cout << "=== SHARED PRINTER SIMULATION ===\n\n";

    // Users submitting jobs
    officePrinter.addJob(101, "Engr. Cenndy", 6);
    officePrinter.addJob(102, "Engr. Cinndy", 7);
    officePrinter.addJob(103, "Engr. Cendy", 19);
    officePrinter.addJob(104, "Engr. Cindy", 5);

    // View queue before processing
    officePrinter.showPendingJobs();

    // Process all jobs
    officePrinter.processJobs();

    // Verify empty queue
```

```
            pendingCount--;
        }
        backPtr = nullptr; // Reset the back pointer once queue is empty
        std::cout << "All print jobs completed successfully.\n";
    }

    // Display all jobs currently waiting
    void showPendingJobs() const {
        std::cout << "\n--- CURRENT PENDING JOBS (" << pendingCount << ") ---\n";
        if (frontPtr == nullptr) {
            std::cout << "Printer queue is empty.\n";
            return;
        }
        JobNode* curr = frontPtr;
        while (curr != nullptr) {
            std::cout << " -> ";
            curr->job.displayJob();
            std::cout << "\n";
            curr = curr->next;
        }
    }
};

// Step 4: Scenario Simulation
int main() {
    Printer officePrinter;

    std::cout << "=== SHARED PRINTER SIMULATION ===\n\n";

    // Users submitting jobs
    officePrinter.addJob(101, "Engr. Cenndy", 6);
    officePrinter.addJob(102, "Engr. Cinndy", 7);
    officePrinter.addJob(103, "Engr. Cendy", 19);
    officePrinter.addJob(104, "Engr. Cindy", 5);

    // View queue before processing
    officePrinter.showPendingJobs();

    // Process all jobs
    officePrinter.processJobs();

    // Verify empty queue
    officePrinter.showPendingJobs();

    return 0;
}
```

Outline
Timeline

Output:

```
compilation terminated.
PS C:\Users\joyni\OneDrive\Desktop\job> g++ main.cpp -o program
PS C:\Users\joyni\OneDrive\Desktop\job> ./program
=== SHARED PRINTER SIMULATION ===

[ADDED] Job #101 | User: Engr. Cenndy | Pages: 6
[ADDED] Job #102 | User: Engr. Cinndy | Pages: 7
[ADDED] Job #103 | User: Engr. Cendy | Pages: 19
[ADDED] Job #104 | User: Engr. Cindy | Pages: 5

--- CURRENT PENDING JOBS (4) ---
 -> Job #101 | User: Engr. Cenndy | Pages: 6
 -> Job #102 | User: Engr. Cinndy | Pages: 7
 -> Job #103 | User: Engr. Cendy | Pages: 19
 -> Job #104 | User: Engr. Cindy | Pages: 5

--- STARTING PRINTER JOB PROCESSING ---
[PROCESSING] Job #101 | User: Engr. Cenndy | Pages: 6 -> Completed!
[PROCESSING] Job #102 | User: Engr. Cinndy | Pages: 7 -> Completed!
[PROCESSING] Job #103 | User: Engr. Cendy | Pages: 19 -> Completed!
[PROCESSING] Job #104 | User: Engr. Cindy | Pages: 5 -> Completed!
All print jobs completed successfully.

--- CURRENT PENDING JOBS (0) ---
Printer queue is empty.
```

This program system simulates a shared printer system using a queue implemented with a link list The queue follows the First-Come, First-Served (FCFS) ; it means the first job will be printed and added to the queue that's the first processed. The program demonstrates the print job. It contains the three data the jobID for identifying the print job, userName for storing the name of the user who submitted the job, and numberOfPages for storing the name of pages to be printed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Just for clarification and explain some of my questions and also for translator**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.